Time and Space Complexity

25 February 2026 19:57

Two criteria are used to judge algorithms:
1. Time complexity
2. Space complexity

Time complexity of an algorithm is the amount of CPU time it needs to run completion.

Space complexity of an algorithm is the amount of memory it needs to run completion.

TIME:
-Operations
-Comparisons
-Loops
-References
-Function calls to outside

SPACE:
-Variables
-Data Structures(Array, LinkedList Tree, etc.)
-Allocations
-Function call (Memory STACKs)

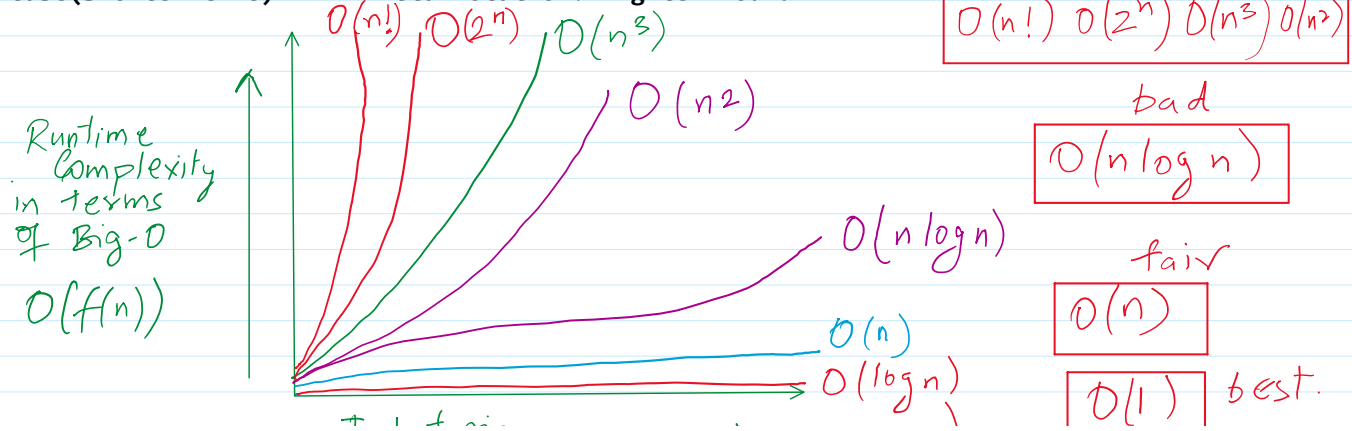
What is time complexity?
Time complexity of an algorithm is the amount of time(or number of steps) needed by a program to complete its task(to execute a particular algorithm)

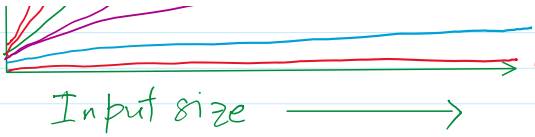
The time taken for an algorithm is comprised of two times:
1. Compilation time
2. Run time

Compile time:
- Compilation time is the time taken to compile an algorithm
- While compiling it checks the syntax and semantic errors in the program and it links it with the standard libraries.

Run time:
- It is a time to execute the compiled program
- The runtime of an algorithm depends on the number of instructions present in the algorithm
- Note that the runtime is calculated only for executable statements and not for declaration statements

Types of time complexity
1. Worst Case(longest time) **Big Oh Notation: Upper Bound**
2. Average Case(Average time) **Omega Notation: Lower Bound**
3. Best Case(Shorter time) **Theta Notation: Tighter Bound**





O(1) best.

in fraction time complexity.

```
boolean isPrime(int x){
    int i, count=0;
    for(i=1; i<=x; i++){
        if(x%i==0){
            count++;
        }
    }
    if(count==2){
        return true;
    }else{
        return false;
    }
}
```

x = 11357962 O(x)

The number which divisible by 1 and itself only, then it is a prime number. For example, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29,

```
boolean isPrime(int x){
    if(x>2 && x%2==0)
        return false;
    for(int od=3; od<=Math.sqrt(x); od+=2){
        if(x%od==0){
            return false;
        }
    }
    return true;
}
```

worst case complexity

$O\left(\frac{\sqrt{n}}{2}\right)$

$\frac{10}{2}$ Time complexity

Standard Analysis Techniques

- **Constant time statements**
- **Analyzing Loops**
- **Analyzing Nested Loops**
- **Analyzing Sequence of statements**
- **Analyzing Conditional statements**

← Time and space are dependent on these analysis.

Space Complexity of Algorithm

Space Complexity of a program is the amount of memory consumed by the algorithm until it completes its execution.

The space occupied by the program is generally by the following:

- A fixed amount of memory occupied by the space for the program, i.e, data types.
- Code and space occupied by variables used in the program.

- A variable amount of memory occupied by the component variable whose size is dependent on the problem being solved.
- The space increases or decreases depending on whether the program uses iterative or recursive procedures.

Space Complexity = Auxiliary Space + Input Space

Types of Space Complexity

1. A **fixed part**, that is a space required to store certain data and variables that are independent of the size of the problem. For example, simple variables and constant used in the program, etc.
2. A **variable part** is a space required by variables whose size depend on the size of the problem. For example, dynamic memory allocation, recursion, stack space etc.

Space complexity S(P) of any algorithm P is S(P) = C + S(I)

Here C is the fixed part and S(I) is the variable part.

Other types of space

Instruction Space: is the space in memory occupied by the compiled version of the program. We consider this space as a constant space for any value of N. The instruction space is independent of the size of the problem.

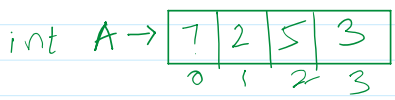
Data Space: is the space and memory which is used to hold the variables, data structures, allocated memory and other data elements. The data space is related to the size of the problem.

Environment Space: is the space in memory used on runtime stack for each function call. This is related to the runtime stack and holds the returning address of the previous function. Stored return value and pointer on it.\

Example of Space complexity

Java Programming language

Type	Size
boolean	1 bit
byte	1 byte
char	2 bytes
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes




```
int sum(int A[]){
   int i, s = 0 ;
   for(i = 0; i < A.length; i++){
      s += A[i];
   }
   return s;
}
```

$S(P) = S(I) + C$

$4 \times 4 + 4 + 4 = 24$ bytes

Difference between Space Complexity and Time Complexity

Space Complexity	Time Complexity
Space complexity is the space (memory) needed. For an algorithm to solve the problem. An efficient algorithm takes space as small as possible.	Time complexity is the time required for an algorithm to complete its process. It allows comparing the algorithm to check which one is the efficient one.

Time complexity of a program is a simple measurement of how fast the time taken by the program grows if the input increases.

Space complexity of a program is a simple measurement of how fast the space taken by a program grows if the input increases.

Calculations in different cases

1. Loop

```
for (i = 0; i < n; i++){  // n
   r = a + b;   //constant time
}

Time Complexity: O(n)
```

2. Nested loop

```
for (i = 0; i < n; i++){  // n
   for(j = 0; j < n; j++){ // n
      r = a + b;   //constant time
   }
}

Time Complexity: O(n^2)
```

- **Constant time can be neglected**

3. Sequential Statements

```
a = a + 1; //constant time
for (i = 0; i < n; i++){  // n
   r = a + b;   //constant time
}
for (i = 0; i < n; i++){  // n
   y = a + b;   //constant time
}

O(n) = c1 + c2.n + c3.n = n (major term)
Time complexity: O(n)
```

4. If-else Statement

```
if(condition){
   … // t
}
else{
   … // t
}

Time complexity: O(t) which is a constant.
```

if else statement contains loops

```
if(condition){
   for (i = 0; i < n; i++){  // n
      r = a + b;   //constant time
   }
}
else{
   for (i = 0; i < n; i++){  // n
      r = a + b;   //constant time
   }
}
```

Time complexity: O(n)

if else statement contains loops and nested loops

```
if(condition){
   for (i = 0; i < n; i++){  // n
      r = a + b;   //constant time
   }
}
else{
   for (i = 0; i < n; i++){  // n
      for(j = 0; j < n; j++){ // n
         r = a + b;   //constant time
      }
   }
}
```

Time complexity: $O(n^2)$

5. **Loops running n times and incrementing or decrementing by constant factor**

```
c = 2;
for (i = 1; i <= n; i*= c){  // c is some constant factor
   r = a + b;   //constant time
}
```

Time complexity: O(log n)

i -> 1, 2, 4, 8, 16, 32,…2^k

$n = 2^k \Rightarrow 2^k = n$ (apply $\log_2$ both side)

$\log 2^k = \log n$ ← constant factor

$k(\log_2 2) = \log n$

$k = \log n$.

Consider this as one quantity.

1, 5, 25, 125, ... 5^k

```
int i;
for(i=1; i<(n*n*n); i*=5){
   cout<<"Hello!";
}
```

constant factor

$5^k = n^3$

$\log_5 5^k = \log_5 (n^3)$

```
    Sopln("Hello!");
}
```

constant factor

$\log_5 5^K = \log_5 (n^3)$

$K \log_5 5 = \log_5 n^3$

$K = O(\log_5 (n^3))$

```
{
  int i=1, k=1
  while (k<=n){
    i++;
    k += i;
    Sopln("Hi!");
  }
}
```

// constant time.

i	1	2	3	4	5	6	....	n
k	1	3	6	10	15	21	....	z

$1+2+3+4+5+6 = 21$

$\frac{z(z+1)}{2} = n$

$z^2 + z = 2n$

major term

$\sqrt{z^2} = \sqrt{n}$

here we are not considering the minor terms & constants

$z = O(\sqrt{n})$

$\frac{3 \times 7}{2} = 21$

Home work

Q1.

```
i,j,k;
for(i=n/2; i<=n; i++){
  for(j=1; j<=n/2; j++){
    for(K=1; K<=n; K*=2){
      Sopln("Hello!");
    }
  }
}
```

Q2

```
int i=n;
while(i>1){
  Sopln("Hello!");
  i/=2;
}
```

Q3

```
int i,j,K;
for(i=n/2; i<=n; i++){
  for(j=1; j<=n; j*=2){
    for(K=1; K<=n; K*=2){
      Sopln("Hello!");
    }
  }
}
```